

Data Driven Testing

Certainly! Below are the **steps to implement Data-Driven Testing** using **CSV files** in **Python (with PyTest)**, **Java (with TestNG)**, and **JavaScript (with Cypress)**. Each section will describe the steps for reading, manipulating, and passing data to your tests.

Steps for Implementing Data-Driven Testing with CSV Files

1. Python (PyTest)

Step 1: Set Up Your Environment

Install PyTest (if not already installed):

- `pip install pytest`

Install `pandas` or `openpyxl` (for Excel) if you're going to manipulate complex data:

- `pip install pandas openpyxl`

Step 2: Prepare Your CSV Data

Create a CSV file (e.g., `login_data.csv`) with data like:

- `username,password`
- `user1,pass1`
- `user2,pass2`
- `user3,pass3`

Step 3: Write a Function to Read CSV Data

- Using Python's built-in `csv` module or `pandas` to read data from the CSV file:

Using `csv` module:

```
o import csv
o
o def read_csv(file_name):
o     data = []
o     with open(file_name, mode='r') as file:
o         csv_reader = csv.reader(file)
o         for row in csv_reader:
o             data.append(row)
o     return data
```

Using `pandas` for easier data manipulation:

```
o import pandas as pd
```

```

o
o def read_csv(file_name):
o     return pd.read_csv(file_name)

```

Step 4: Use `pytest.mark.parametrize` to Inject Data

Parametrize the test with the CSV data:

```

• import pytest
• import csv
•
• @pytest.mark.parametrize("username, password",
•     read_csv("login_data.csv"))
• def test_login(username, password):
•     print(f"Testing login with username: {username} and password:
•         {password}")
•     # Add Selenium code here to perform login using username and
•     password

```

Step 5: Run the Test

Run the test with PyTest:

```

• pytest test_script.py

```

Step 6: Manipulate Data (Optional)

You can manipulate the data before passing it to the test function, such as filtering or updating values:

```

• def filter_data(data):
•     return [row for row in data if row[0] == "valid_user"]
•
• filtered_data = filter_data(read_csv("login_data.csv"))

```

2. Java (TestNG)

Step 1: Set Up Your Environment

Install TestNG via Maven in your pom.xml:

- `<dependency>`
- `<groupId>org.testng</groupId>`
- `<artifactId>testng</artifactId>`
- `<version>7.5.0</version>`
- `<scope>test</scope>`
- `</dependency>`
- **Install OpenCSV (for reading CSV data) via Maven:**
- `<dependency>`
- `<groupId>com.opencsv</groupId>`
- `<artifactId>opencsv</artifactId>`
- `<version>5.6</version>`
- `</dependency>`

Step 2: Prepare Your CSV Data

Create a CSV file (e.g., login_data.csv) with data like:

- username,password
- user1,pass1
- user2,pass2
- user3,pass3

Step 3: Create a DataProvider Method to Read CSV

Use **OpenCSV** to read and return data from CSV for TestNG:

- `import com.opencsv.CSVReader;`
- `import org.testng.annotations.DataProvider;`
- `import org.testng.annotations.Test;`
-
- `import java.io.FileReader;`
- `import java.util.ArrayList;`
- `import java.util.List;`
-
- `public class DataDrivenTest {`
-
- `@DataProvider(name = "loginData")`
- `public Object[][] getLoginData() throws Exception {`
- `List<String[]> data = new ArrayList<>();`
- `CSVReader reader = new CSVReader(new`
- `FileReader("login_data.csv"));`
- `String[] nextLine;`
- `while ((nextLine = reader.readNext()) != null) {`
- `data.add(nextLine);`
- `}`
- `return data.toArray(new Object[data.size()][]);`

- }
-
- @Test(dataProvider = "loginData")
- public void testLogin(String username, String password) {
- System.out.println("Testing login with username: " + username
- + " and password: " + password);
- // Add Selenium code here to perform login using username and
- password
- }
- }

Step 4: Run the Test

Run the TestNG test as usual from your IDE or via Maven:

- mvn test

Step 5: Manipulate Data (Optional)

If needed, manipulate or filter the data before passing it to the test:

- public Object[][] getFilteredLoginData() throws Exception {
- List<String[]> data = new ArrayList<>();
- CSVReader reader = new CSVReader(new
- FileReader("login_data.csv"));
- String[] nextLine;
- while ((nextLine = reader.readNext()) != null) {
- if ("valid_user".equals(nextLine[0])) {
- data.add(nextLine);
- }
- }
- return data.toArray(new Object[data.size()][]);
- }

3. JavaScript (Cypress)

Step 1: Set Up Your Environment

- Install Cypress:
- `npm install cypress`
- Install **PapaParse** for CSV parsing:
- `npm install papaparse`

Step 2: Prepare Your CSV Data

- Create a CSV file (e.g., `login_data.csv`) with data like:
- `username,password`
- `user1,pass1`
- `user2,pass2`
- `user3,pass3`

Step 3: Load CSV Data Using `cy.fixture()`

- Place your CSV file under `cypress/fixtures/` folder.

Step 4: Write a Test That Uses CSV Data

Use **PapaParse** to parse the CSV file and manipulate data:

```
• import Papa from 'papaparse';
•
• describe('Login Test', () => {
•   let loginData;
•
•   before(() => {
•     cy.fixture('login_data.csv').then((csvData) => {
•       loginData = Papa.parse(csvData, { header: true }).data;
•       // Manipulate or filter data if necessary
•       loginData = loginData.filter(row => row.username !==
'invalid_user');
•     });
•   });
•
•   loginData.forEach((data) => {
•     it(`should login with username: ${data.username}`, () => {
•       cy.visit('/login');
•       cy.get('input[name="username"]').type(data.username);
•       cy.get('input[name="password"]').type(data.password);
•       cy.get('button[type="submit"]').click();
•       // Replace with actual assertions
•     });
•   });
• });
```

Step 5: Run the Test

Run Cypress tests:

- `npx cypress open`

Step 6: Manipulate Data (Optional)

Filter, update, or manipulate data:

- `loginData = loginData.map(row => {`
 - `row.password = row.password + '_updated'; // Append '_updated' to`
 `passwords`
 - `return row;`
 - `});`
-

General Steps for Data-Driven Testing (CSV)

1. **Set Up Environment:** Install necessary libraries and frameworks.
2. **Prepare Data:** Organize your test data in a CSV file with appropriate columns (e.g., username, password).
3. **Read Data:** Use libraries like `csv`, `pandas`, `OpenCSV`, or `PapaParse` to read the CSV file into a data structure (e.g., list, array).
4. **Manipulate Data:** (Optional) Filter, transform, or update the data to meet specific test needs.
5. **Inject Data into Tests:** Use parametrization (`@pytest.mark.parametrize`, `@DataProvider`, `cy.fixture`) to pass the data into your test methods.
6. **Run Tests:** Execute the tests with the injected data and ensure that results are validated correctly.

These steps will help you implement **Data-Driven Testing** effectively in **Python (PyTest)**, **Java (TestNG)**, and **JavaScript (Cypress)** using CSV files. Let me know if you need further details!